

Unidad 1

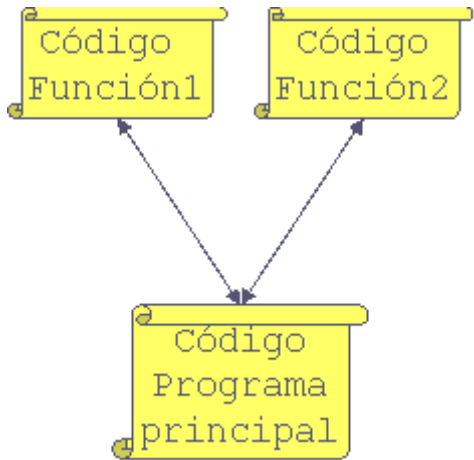
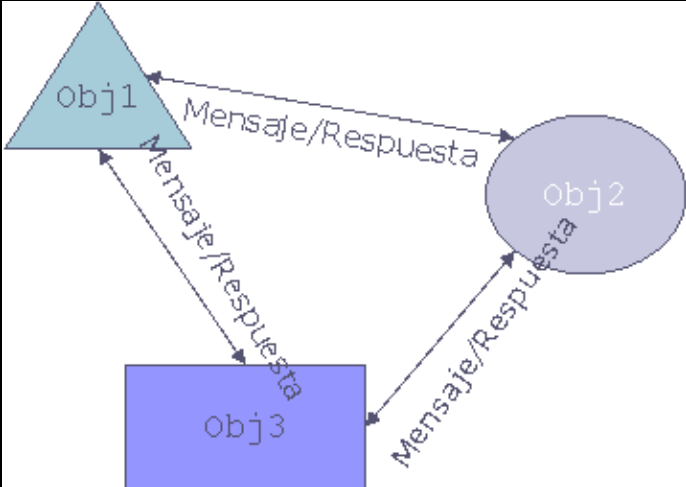
PROGRAMACIÓN ORIENTADA A OBJETOS-POO

1. Conceptos Básicos

1.1. Introducción

La POO trata de amoldarse al modo de pensar del hombre y no al de la máquina. Esto es posible gracias a la forma racional de manejar sus propiedades como son: las abstracciones, la jerarquía, el encapsulamiento y el polimorfismo. El elemento básico de este paradigma no es la función (elemento básico de la programación estructurada), sino un ente denominado objeto. Un objeto es la representación de un concepto para un programa, y contiene toda la información necesaria para abstraer dicho concepto, que son: los datos (variables) que describen su estado y las operaciones (métodos) que pueden modificar dicho estado o determinar las capacidades que posee dicho objeto, y de esta forma trata de asemejarse al mundo real.

Existen muchos paradigmas de programación; los más generales son: el Lineal, el Modular, el estructurado, el Orientado a Objetos y el Orientado a Aspectos.

COMPARACIÓN	
Programación Estructurada: El elemento principal es la función (o Subprograma). Los datos (variables) se diferencian claramente de los procesos (métodos) y no existe control de acceso a ellos.	Programación Orientada a Objetos: El elemento principal es el Objeto. Los datos (variables) y los procesos (métodos) forman una entidad única llamada objeto.
	

1.2. OBJETO:

Podemos definir un objeto como una estructura de programación **dinámica**, con un conjunto de operaciones (o métodos) y un estado (o variables). Por esto se dice que en la programación orientada a objetos "se unen datos y procesos", y no como en su predecesora, la programación estructurada, en la que estaban separados en forma de variables y funciones.

Un objeto es dinámico porque consta de:

- ✓ **Tiempo de vida:** La duración de un objeto en un programa siempre está limitada en el tiempo. La mayoría de los objetos sólo existen durante una parte de la ejecución del programa. Los objetos son creados mediante un mecanismo denominado instanciación, y cuando dejan de existir se dice que son destruidos.
- ✓ **Estado:** Todo objeto posee un estado, definido por sus atributos (Variables). Con él se definen las propiedades del objeto, y el estado en que se encuentra en un momento determinado de su existencia.

- ✓ **Comportamiento:** Todo objeto ha de presentar una interfaz, definida por sus métodos, para que el resto de objetos que componen los programas puedan interactuar con él.

El equivalente de un Objeto en el paradigma estructurado sería una variable, pero con comportamiento, porque incluye métodos. Así mismo la instanciación de un objeto equivaldría a la declaración de una variable y el tiempo de vida de un objeto el ámbito de la variable.

1.3. Principios de la POO

Existen una serie de principios fundamentales para comprender cómo se modeliza la realidad al crear un programa bajo el paradigma de la orientación a objetos. Estos principios son: Abstracción, Encapsulamiento, Modularidad, Jerarquía, Paso de Mensajes y Polimorfismo.

Principio de abstracción

Mediante la abstracción la mente humana modeliza la realidad en forma de objetos. Para ello busca semejanzas entre la realidad y la implementación de objetos en el programa, que simulen el funcionamiento de los objetos reales.

Los seres humanos no pensamos en los objetos reales como un conjunto de cosas menores; por ejemplo, no vemos un cuerpo humano como un conjunto de células y órganos con funciones propias. Las personas entendemos la realidad como objetos con comportamientos bien definidos. No necesitamos conocer los detalles de porqué, ni cómo funcionan las cosas; simplemente solicitamos determinadas acciones en espera de una respuesta. Lo mismo busca la POO, al programador promedio no le interesa como se obtiene un número aleatorio, solo instancia u obtiene y lo utiliza.

Principio de encapsulamiento

El encapsulamiento permite a los objetos elegir qué información es publicada o utilizables y que información es ocultada al resto de los objetos. Para ello los objetos suelen presentar sus métodos como interfaces públicas y sus atributos como datos privados e inaccesibles desde otros objetos. De esta manera el acceso a los datos de los objetos es controlado por el programador, evitando efectos laterales no deseados. Con el encapsulado de los datos se consigue que las personas que utilicen un objeto sólo tengan que comprender su interfaz o métodos, olvidándose de cómo está implementada, y en definitiva, reduciendo la complejidad de utilización.

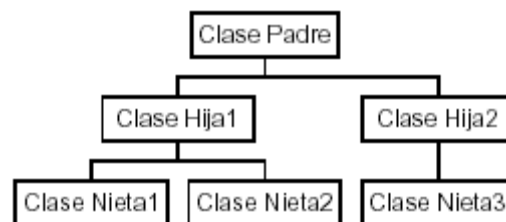
Principio de modularidad

Mediante la modularidad, se propone al programador dividir su aplicación en varias partes o módulos diferentes (ya sea en forma de clases, paquetes o bibliotecas), cada uno de ellos con un sentido propio. Esta fragmentación disminuye el grado de dificultad del problema al que da respuesta el programa, pues se afronta el problema como un conjunto de problemas de menor dificultad, facilitando la comprensión del programa (en concreto se basa en la máxima divide y vencerás).

Principio de Jerarquía

La mayoría de nosotros ve de manera natural nuestro mundo como objetos que se relacionan entre sí de una manera jerárquica. Por ejemplo; un municipio es de un departamento, un departamento es de un país, un país es de un continente...

Del mismo modo, las distintas clases de un programa que poseen una relación, se organizan mediante la Jerarquía o Herencia. La representación de dicha organización da lugar a los denominados árboles herencia. Ejemplo:



Mediante la Herencia la clase hija2 (sub clase) puede tomar variables y métodos de una clase padre (clase base o súper clase). Así se simplifican los diseños y se evita la duplicación de código; al no tener que volver a escribir métodos y variables ya implementados. Al acto de tomar propiedades de una clase padre se denomina heredar.

Principio del paso de mensajes

Mediante el denominado paso de mensajes, un objeto puede solicitar de otro objeto que realice una acción determinada o que modifique su estado. El paso de mensajes se suele implementar como llamadas

a los métodos de otros objetos. Desde el punto de vista de la programación estructurada, esto correspondería con la llamada a funciones.

Principio de polimorfismo

Polimorfismo quiere decir "un objeto y muchas formas". Esta propiedad permite que un objeto presente diferentes comportamientos en función del contexto en que se encuentre. Por ejemplo, un método puede presentar diferentes implementaciones en función de los argumentos que recibe, recibir diferentes números de parámetros para realizar una misma operación o realizar diferentes acciones dependiendo del nivel de abstracción en que sea llamado.

1.4. PAQUETE:

Los paquetes, denominados en C# como "[namespace](#)", son el mecanismo que permite agrupar clases e interfaces con algún tipo de relación en común. Con la idea de mantener la reutilización, organización y facilidad de uso de los paquetes desarrollados es conveniente que las clases e interfaces contenidas en los mismos tengan cierta relación funcional. De esta manera los desarrolladores ya tendrán una idea de lo que están buscando y fácilmente sabrán qué pueden encontrar dentro de un paquete. Una solución o aplicativo puede poseer varios paquetes.

1.5. INTERFACES:

Es una estructura similar a una clase, permiten declarar métodos abstractos (sin cuerpo), que posteriormente pueden ser implementados de diferentes maneras en la clase que implemente la interface, según las necesidades de un programa. Se crean similar a una clase, solo que en la declaración se cambia el comando [class](#) por [interface](#).

2. CLASES

2.1. Introducción

Una clase es una estructura de programación **estática** que agrupa de datos (variables) y de funciones (métodos) que operan sobre esos datos. La clase es la estructura fundamental de programación en la POO y posee tres funcionalidades básicas que son: como un programa en sí (es decir ejecutable, por poseer el método Main), como una clase de propósito general (colección de métodos relacionados, que no es ejecutable) o como un molde o prototipo para la creación de Objetos.



Un ejemplo de una clase de propósito general es la clase Leer; la cual simplifica la lectura desde el teclado de variables primitivas: [short](#) (entero corto), [int](#) (entero), [long](#) (entero largo), [float](#) (real corto), [double](#) (real largo), [char](#) (carácter) y [string](#) (cadena), para cada tipo se implementa un método con un pequeño control sobre la información digitada.(ver al final anexo)

Nota: La clase Leer debe estar en el mismo directorio o carpeta de las clases que la van a utilizar (si no se debe importa con el comando [using](#)) y por sí solo no realiza nada, ya que no posee método principal (Main).

Ejemplo para probar la clase Leer:

```
public class LeerDatos {
    public static void Main(string[] args) {
        //Declaración de variables
        char var_caracter = '\0';
        short var_short = 0;
        int var_int = 0;
        long var_long = 0;
        float var_float = 0;
        double var_double = 0;
        string var_cadena = "";
        //Instrucciones de lectura
        System.Console.WriteLine("Dato char: ");
        var_caracter = Leer.caracter(); //Llama al método caracter de la clase Leer
    }
}
```

```

System.Console. Write ("Dato short: ");
var_short = Leer.datoShort(); //Llama al método datoShort de la clase Leer
System.Console. Write ("Dato int: ");
var_int = Leer.datoInt(); //Llama al método datoInt de la clase Leer
System.Console. Write ("Dato long: ");
var_long = Leer.datoLong(); //Llama al método datoLong de la clase Leer
System.Console. Write ("Dato float: ");
var_float = Leer.datoFloat(); //Llama al método datoFloat de la clase Leer
System.Console. Write ("Dato double: ");
var_double = Leer.datoDouble(); //Llama al método datoDouble de la clase Leer
System.Console. Write ("Dato String: ");
var_cadena = Leer.dato(); //Llama al método dato de la clase Leer
//Instrucciones para imprimir los datos leídos
System.Console.WriteLine(" LOS DATOS LEIDOS SON:");
System.Console. WriteLine (var_caracter);
System.Console. WriteLine (var_short);
System.Console. WriteLine (var_int);
System.Console. WriteLine (var_long);
System.Console. WriteLine (var_float);
System.Console. WriteLine (var_double);
System.Console. WriteLine (var_cadena);
}
}

```



Práctica 1: Hacer una clase de propósito general, similar a la clase Leer, para realizar las operaciones matemáticas más comunes (suma, resta, multiplicación, división, modulo, elevar al cuadrado, potenciación, raíz cuadrada, radicación, seno, coseno y tangente en grados) y otra clase que permita probar la clase de las operaciones matemáticas.

2.2. Algunas características importantes de las clases

1. Todas las variables y métodos de C# deben pertenecer a una clase. No hay variables y metodos globales.
2. Si una clase deriva de otra (:), hereda todas sus variables y métodos miembro.
3. C# tiene una jerarquía de clases estándar de la que pueden derivar las clases que crean los usuarios.
4. Una clase sólo puede heredar de una única clase (en C# no hay herencia múltiple). Si al definir una clase no se especifica de qué clase deriva, por defecto la clase deriva de [Object](#). La clase [Object](#) es la base de toda la jerarquía de clases de C#.
5. En un archivo se pueden definir varias clases, pero lo habitual es escribir una sola clase por archivo.
6. Las clases se pueden agrupar en [namespace](#) o paquetes, introduciendo una línea al comienzo del archivo ([namespace](#) NombrePaquete;). Esta agrupación en paquetes está relacionada con la jerarquía de directorios y ficheros en la que se guardan las clases.
8. Los métodos y variables de una clase pueden referirse de modo general al objeto de esa clase por medio de la referencia [this](#). y a los métodos y variables de la superclase de la que hereda con la referencia [base](#).

2.3. Formato General de un Clase

La implementación de una clase comprende dos partes: la declaración y el cuerpo:

Declaración { cuerpo de la clase }

FORMATO DE LA DECLARACIÓN DE LA CLASE

[Acceso] [Característica] **class** NombreClase [: NombreSuperClase] [: NombreInterface]

DONDE:

[Acceso]: Determina si la clase es **public**, **private** o **protected**.

[Característica]: Determina comportamientos especiales a las clases. Por ejemplo: **abstract**; (abstracta) implica que posee métodos sin implementar (sin cuerpo o abstractos), no se pueden crear objetos de ella y debe ser una superclase, para que en las clases hijas se implementen dichos métodos. **sealed**; que no permite herencia, es decir, no puede tener clases derivadas o subclases. **static**; que de esta clase no se pueden crear objetos. **partial**: Que la clase está dividida en varios archivos.

NombreClase: Corresponde a un identificador válido en C# y por convención debe iniciar con mayúscula.

[: NombreSuperClase]: (Herencia) define que es una subclase y que deriva o hereda de una superclase.

[: NombreInterface]: (implementa) determina que interface o interfaces implementan la clase, una interface es una especie de clase que solo posee definiciones de métodos y constantes.

EJEMPLO:

ÁRBOL DE HERENCIA	EJEMPLOS
<pre>graph TD; Padre[Clase Padre] --> Hija1[Clase Hija1]; Padre --> Hija2[Clase Hija2]; Hija1 --> Nieta1[Clase Nieta1]; Hija1 --> Nieta2[Clase Nieta2]; Hija2 --> Nieta3[Clase Nieta3];</pre>	class Padre { . . }: Es la forma más simple, declara un clase llamada Padre, deriva por defecto de la clase Object de C# y no implementa ninguna interface. public class Padre { . . }: Lo mismo que la anterior, pero con la diferencia que es pública. class Hija1 : Padre { . . }: Declara la subclase hija1 que deriva o hereda de la clase Padre. abstract class Hija2 : Padre { . . }: Declara la subclase Hija2 que deriva de la Clase Padre y que posee métodos solamente declarados o abstractos y que dichos métodos se deben implementar en la clase Nieta3 para poder crea objetos de esta. sealed class Nieta1 : Hija1 { . . }: Declara la subclase Nieta1, que deriva de Hija1 y que de esta clase no pueden heredar otras clase. class Nieta2 : Hija1 : Padrino { . . }: Declara la subclase Nieta2 que deriva de Hija1 y utiliza la interface Padrino.

FORMATO DEL CUERPO DE LA CLASE

```
{ //inicio clase
  Declaración de Variables Miembro
  Declaración de Método1 {
    Declaración de variables Locales metodo1
    Conjunto de sentencias metodo1
  }

  Declaración de Clase Interna {
    Declaración de Variables Miembro de la clase Interna
    Declaración de Métodos de la clase Interna {
      Declaración de variables Locales de la clase Interna
      Conjunto de sentencias
    }
  }
}
```

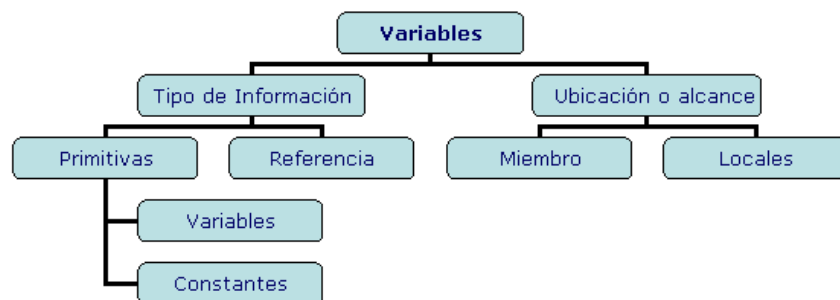
```
Declaración de MétodoN { . . . }  
} //fin clase
```

NOTA: Una clase puede contener otras clases, a estas clases se les denomina clases internas y su comportamiento es similar a una clase normal. Además, tiene visibilidad de los métodos y de las variables miembro de la clase que las contienen.

3. VARIABLES

3.1. Tipos de Variables

Ya se han tratado el tema de las variables de C# al principio del curso, pero falta ampliar el concepto para adaptarlo a la POO.



3.2. Variables Tipo Referencia

Son las que identifican a los objetos, a este tipo pertenecen las variables de cadena **String** (no string con minúscula) o **Double** (no double) y las variables de los arreglos (vectores, matrices) que son considerados objetos en C#. Para darles un nombre o identificador se siguen las mismas reglas que se mencionaron para las variables primitivas. Son apuntadores a espacios de memoria donde esta almacenada la información del objeto (Variables primitivas y métodos), de ahí su nombre; hacen referencia a un espacio de memoria.

3.3. Variables Miembro

Pueden ser de tipo primitivo o de referencia a objetos, y su declaración debe aparecer dentro del cuerpo de la clase, las que están al interior del cuerpo de un método o en un bloque (por ejemplo: de una instrucción de ciclo o decisión) son variables locales y su existencia se limita al bloque en que se definen. Cada que se crea un objeto el compilador hace una copia de las **variable miembros de la clase** para es objeto, a estas variables se les llaman **variables miembro del objeto** y a las variables que en su declaración se colocan **static** son variables miembro de la clase o propias de la clase y se utilizan para definir variables comunes a todos los objetos de la clase, no se hacen copias para los objetos de esta clase y son lo más parecido a las variables globales de C++ y de Visual Basic.

3.4. Formato General para Declarar una Variable:

[Acceso] [Característica] Tipo NombreVariable [= Valor]

DONDE:

[Acceso]: Junto con los modificadores de acceso de la clase, determinan que clases y métodos van a tener permiso para utilizar la variable miembro. Los modificadores de acceso son: **public**; puede ser usada por cualquier clase. **private**; Solo puede ser usada por la clase donde se declara. **protected**; usan la variable la familia de la clase donde se declara.

[Característica]: Les da una propiedad especial a las variables y pueden ser: **static** (estática) indica que la variable es una variable miembro de la clase. **const** que la variable es un constante, ósea que su valor no se puede modificar. **readonly** (solo lectura) declara una variable a la que solo se pueden asignar valores como parte de la declaración o en un constructor de la misma clase. la variable no es un parte persistente del estado del objeto, es decir, su valor no se puede guardar como las demás variables miembro del objeto. Y **volatile** (volátil) significa que la variable se modifica de forma asincrónica o que puede ser modificada por el sistema operativo, el hardware o otros programas en ejecución simultanea (multiproceso o multihilos).

Tipo: Si es primitiva se coloca la palabra clave primitiva (por ej. **char**, **int**, **double**, etc.) y si es referencia a objetos se coloca el nombre de la clase que se instancia o que sirve de molde para crea el objeto.

La parte **NombreVariable [= Valor]**: es igual a la ya tratada al principio.

4. MÉTODOS

Con los métodos curre algo similar a lo del tema anterior, hay que ajustarlos a la POO. En ésta parte ampliara los conceptos de: formato de los métodos y los tipos de métodos desde la POO.

4.1. Formato General para Declarar un Método:

```
[Acceso] [Característica] TipoValorRetorno NombreMétodo ([lista de parámetros]) {  
    Cuerpo de Método  
    [return valorRetorno;] }
```

DONDE:

[Acceso]: Similar a los modificadores de acceso del as variables (**public**, **prívate** y **protected**), con los modificadores de acceso de la clase y de las variables conforman la propiedad de la POO encapsulamiento; que consiste en impedir el uso indebido de las variables, métodos y clases, implicando un alto grado de seguridad en C#. En la tabla siguiente se resume los niveles de acceso permitidos por cada modificador:

ACCESO A	private	protected	public
La clase	SI	SI	SI
Clase del paquete	NO	NO	SI
Subclase del paquete	NO	SI	SI
Subclase del diferente paquete	NO	SI	SI
Clase de diferente	NO	NO	SI

[Característica]: Pueden ser **static**: o método miembro de clase, similar a las variables. **abstract**: solo posee declaración y no definición o cuerpo del método, si una clase posee un método **abstract** es obligatorio definir la clase como **abstract** y en cualquier subclase este método debe ser redefinido. **override**: permite que el método pueda ser redefinidos o cambiar el conjunto de instrucciones que lo conforman por una subclase que derive de la clase que lo contiene, por defecto no se permite. **virtual**: declara un método cuya implementación se puede en una clase derivada.

TipoValorRetorno: Los métodos solo retornan un valor (primitivo o de referencia a un objeto) y es necesario declarar el tipo del valor que devuelven, si un método no retorna ningún valor, debe declararse que devuelve `void` (nulo) y no es necesario colocar la palabra clave `return` en el cuerpo del método.

NombreMétodo: El nombre de un método puede ser cualquier identificador legal en C#, similar al nombre de variable y por convención los nombres de los métodos empiezan con minúsculas.

([lista de parámetros]): o argumentos explícitos, son los valores que necesita la función o método para su funcionamiento y pueden ser de tipo primitivo o de referencia a objetos, se colocan entre paréntesis, esperados por comas y especificando su tipo, si no hay parámetros se dejan los paréntesis vacíos. En C# los argumentos se pasan siempre por valor, es decir el método recibe una copia de las variables que se pasan como parámetros, si se modifica ésta, valor de la variable original no se modifica (si es de tipo primitivo, la de tipo referencia a objeto si se modifican). En C# se pueden pasar parámetros primitivos por referencia anteponiendo la palabra clave `ref` o `out` a su declaración, por ejemplo:

```
public void NombreMetodo(int j, ref int i) {...} ➔ j se pasa por valor e i por referencia.
```

4.2. Tipos de Métodos desde la POO

1. Métodos Sobrecargados: Son un conjunto de métodos que comparten el mismo nombre, pero diferentes parámetros. Por ejemplo, supongamos que se ha escrito una clase que puede proporcionar varios tipos de datos (cadena, entero, etc.) En C#, se puede utilizar el mismo nombre para todos los métodos, pasándole un tipo de parámetro diferente a cada uno. Los métodos son diferenciados por la firma del método (la firma está compuesta por el valor de retorno, nombre y la cantidad, tipo y orden de los parámetros), por ejemplo: los métodos `metodoA(string s)` y `metodoA(int i)` son dos métodos distintos y únicos, en cambio los métodos `metodoA(string s)` y `metodoA(string t)` son idénticos y si se declaran en una misma clase dará un error de compilación por duplicar la declaración.

2. Métodos Constructores: Todo método cuyo nombre sea igual al nombre de su clase es un método constructor y tiene una tarea especial que realizar, que consiste en inicializar a los objetos que se construyen de dicha clase. Los constructores solo se pueden llamar con el operador `new`.

3. Métodos Redefinidos: Una subclase puede sobrescribir o redefinir a los métodos de su superclase. El método que redefine la subclase debe tener la misma firma (valor de retorno, nombre y la cantidad, tipo y orden de los parámetros) en la superclase.

Ejemplo de método: Programa para romper por fuerza bruta una clave de 4 caracteres, que utiliza un método para definir la clave y realizar la comparación.

```
public class RevientaClaves {
    //método que rompe la clave
    static bool permiso(string clave) {
        System.Console.WriteLine (" Por favor espere se esta procesando la información.....");
        string probarclave = "";
        char car1 = '0', car2 = '0', car3 = '0', car4 = '0';
        bool encontrada = false;
        do {
            do {
                do {
                    probarclave = "" + car1 + car2 + car3 + car4;

                    if (probarclave.Equals(clave)) {
                        System.Console.WriteLine ("Permiso concedido, su password es; "+ probarclave);
                        encontrada = true;
                    }
                }
                car4++;
            } while (car4 != 'z' && !encontrada);
        }
```



```

        car4 = '0';
        car3++;
    } while (car3 != 'z' && !encontrada);
    car3 = '0';
    car2++;
} while (car2 != 'z' && !encontrada);
car2 = '0';
car1++;
} while (car1 != 'z' && !encontrada);
return encontrada;
} //fin metodo
public static void Main (string[] args) {
    string password="";
    bool encontrada=false;
    System.Console.WriteLine (" *** PROGRAMA CLAVES ** ");
    System.Console.Write ("Por favor digite su clave de seguridad...");
    password = System.Console.ReadLine();
    encontrada = permiso( password);
    if(!encontrada) {
        System.Console.WriteLine ("No fue encontrada, lo sentimos");
    }
    System.Console.ReadKey();
}
}

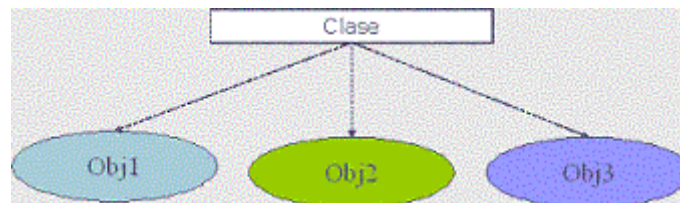
```

5. OBJETOS

5.1. Introducción

Un objeto (en inglés, instance) es una instancia o ejemplo concreto de una clase, son estructura de programación **dinámicas**. Las clases son para los objetos como tipos de variables (char, int, float, etc.), mientras que los objetos son como variables concretas de un tipo determinado, pero con un estado definido por las variables miembro del objeto y un comportamiento dado por los métodos miembro del objeto.

Relación clase-objeto



5.2. Operaciones con Objetos

1. CREACIÓN DE OBJETOS

Es el proceso de declarar una variable tipo referencia y la separación de la memoria necesaria para almacenar la variable y los métodos del objeto. Y se puede realizar de las siguientes formas:

A. Declaración del Objetos: Solo crea la variable tipo referencia al objeto y el asigna un valor de "null".
Formato: NombreClase NombreObjeto;

Ejemplos:

`Circulo` objetoCirculo; // Solo declara un objeto de nombre objetoCirculo de la clase de usuario Circulo

NombreObjeto

`DateTime` hoy; // Declara un objeto de nombre hoy de la clase DateTime de C# que maneja la fecha

B. Separación de Memoria: Este instancia o separa la memoria para el nuevo objeto, que debe estar declarado. Formato: NombreObjeto = `new` NombreMetodoConstructor (Argumentos);

NombreObjeto

Variables del objeto
Métodos del objeto

Ejemplos:

objetoCirculo = `new` Circulo(); //Se asigna memoria el objeto de nombre objetoCirculo

hoy = `new` DateTime(); //Se asigna memoria el objeto de nombre hoy, sin parámetros

fechaCumpleaños = `new` DateTime(1987,08,30); // Se asigna memoria el objeto de nombre fechaCumpleaños, con parámetros

C. Creación del Objeto: Es combina las dos acciones anteriores en un solo línea. Formato:

NombreClase NombreObjeto= `new` NombreMetodoConstructor (Argumentos);

NombreObjeto

Variables del objeto
Métodos del objeto

Ejemplos:

`Circulo` objetoCirculo = `new` Circulo(); //Crea el objeto de nombre objetoCirculo de la clase de usuario Circulo

`DateTime` hoy = `new` DateTime (); //Crea el objeto de nombre hoy de la clase DateTime C#, sin parámetros

`DateTime` fechaCumpleaños = `new` DateTime (1987, 08, 30); //Crea el objeto de nombre fechaCumpleaños de la clase DateTime C#, con parámetros de año, mes y día.

5.3. Manipulación de Objetos

Consiste en emplear las variables y los métodos de un objeto

A. REFERENCIA A UN MÉTODO DEL OBJETO: Se coloca el nombre del objeto un punto (.) y el nombre del método. Formato:

nombreObjeto.nombreMetodo(argumentos);

B. REFERENCIA A UNA VARIABLE DEL OBJETO: Se coloca el nombre del objeto un punto (.) y el nombre de la variable. Formato:

nombreObjeto.nombreVariable;

5.4. Destrucción De Objetos

Cuando un objeto no va a ser utilizado, el espacio de memoria dinámica que utiliza debe ser liberado, así como los recursos que poseía, permitiendo al programa disponer de todos los recursos posibles. A esta acción se la da el nombre de destrucción objetos. En C# la destrucción se puede realizar de dos formas:

A. AUTOMÁTICAMENTE: C# posee un sistema de recogida de basura o Garbage Collector, que por lo general permite que no nos preocupemos de liberar la memoria dinámica asignada a los objetos, que había sido reservada mediante el operador `new`. El sistema de recolección de basura se ejecuta periódicamente, buscando objetos que ya no estén referenciados e inicialmente los marca, para posteriormente liberar su memoria. El recolector puede activarse con el método `System.GC.collect()`. Ejemplo:

```
objetoCirculo = null ; //Se hace perder la referencia al objeto de nombre objetoCirculo.  
System.GC.collect(); //Se invoca al recolector de basura.
```

B. MANUALMENTE: A veces una clase mantiene un recurso que no es de C#, como la apertura de un archivo o una conexión de red o Bases de Datos. En este caso sería acertado el utilizar la destrucción manual de los objetos de dicha clase, para asegurar que dicho recurso se libera. Esto se hace mediante la inclusión en la clase de el método destructor `~Nombreclase()`.

Ejemplo:

```
public class ClaseEjFinalizada { //Clase para crear los objetos  
    ClaseEjFinalizada() { // Método constructor.  
        // Reserva del recurso que no son de C# o recurso compartido  
    }  
    ~ClaseEjFinalizada() { // Liberación del recurso que no son de C# o recurso compartido  
    }  
}
```

C# llama al método `~ClaseEjFinalizada()`, si existe, cuando vaya a reclamar el espacio de memoria de ese objeto, mediante la recogida de basura. Debe observarse que el método `~ClaseEjFinalizada()` es un método redefinido o sobrescrito de la clase `Object` (que es la base de toda las clase en C#)



5.5. Ejercicios de Objetos

1. **CREACIÓN DE OBJETOS EMPLEANDO CLASES DE C#:** C# posee más 3000 de clases que el programador puede emplear para crear objetos de ellas (Por ejemplo, las clases para crear los elementos de la Interface Grafica de Usuario (GUI), para manejar los formatos de números y fecha, para las Cadenas de caracteres, matemáticas, entre otras). En este ejemplo se manejan la fecha y la hora:

```
public class FechaHora {  
    static void imp(string mensaje) { //Método para imprimir  
        System.Console.WriteLine(mensaje);  
    }  
}
```

```

public static void Main(string[] args) {
//Se crea un objeto fecha de la clase DateTime de C# con los valores asignados
    DateTime fecha = new DateTime(2016, 02, 23);
    imp("\t LA FECHA ES: " + fecha.ToString());
    imp("\nAño: " + fecha.Year);
    imp("Mes: " + fecha.Month);
    imp("Día: " + fecha.Day);
    imp("Hora: " + fecha.Hour);
    imp("Minutos: " + fecha.Minute);
    imp("Segundos: " + fecha.Second);
    //Para obtener la fecha actual o del computar se emplea la propiedad Now
    DateTime hoy = DateTime.Now;
    imp("\n \t LA FECHA ACTUAL ES:\n");
    imp("Año: " + hoy.Year + //imprime el año
    "\nMes: " + hoy.Month + //imprime el mes
    "\nDía: " + hoy.Day + //imprime el día
    "\nDía del semana: " + hoy.DayOfWeek + //imprime el día de la semana
    "\nDía del año: " + hoy.DayOfYear + //imprime el día del año
    "\nHora: " + fecha.Hour + //imprime las horas
    "\nMinutos: " + hoy.Minute + //imprime los minutos
    "\nSegundos: " + hoy.Second + "\n"); //imprime los segundos
    System.Console.WriteLine("\n \t Presiones una tecla para terminar...");
    System.Console.ReadKey(); }
}

```

2. CREACIÓN DE OBJETOS EMPLEANDO CLASES DE USUARIO:

A. EJERCICIO DEL CIRCULO: Hacer un programa para crear objetos tipo circulo de la una clase Circulo, en él se deben aplicar los conceptos de métodos constructores y sobrecargados.

B. EJERCICIO PARA EL MANEJO DE LIBROS EN UN BIBLIOTECA

C. EJERCICIO SENCILLO PARA EL MANEJO DE ELECCIONES, para tres candidatos fijos: Clase Candidato (para los objetos) y clase PruebaVotacion

D. OTRO EJERCICIO MÁS ELABORADO PARA EL MANEJO DE ELECCIONES, para hasta 9 candidatos ingresados por el teclado y manejados con un arreglo de objetos: Clase Votación (para los objetos) y clase PruVotacion

E. Hacer un programa que realice una encuesta entre 5 productos de la misma línea de consumo (por ejemplo, refrescos, detergentes, pantalones, etc.). Aplicando Programa Orientado a Objetos.

6. Aplicación de las Propiedades de la POO

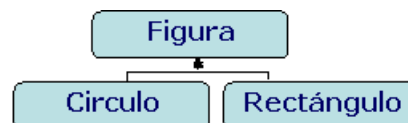
6.1. Herencia

En la POO es la forma de reutilizar código, También se le conoce como derivación y es la capacidad para crear nuevas clases (subclase) a partir de otras existentes (denominadas clase base o superclases, que no se pueden declarar como **sealed**), permitiendo que está le transmita todas sus características (las variables y método miembro). La subclase puede redefinir las propiedades que hereda de la superclase y para referirse a las propiedades de la superclase se debe anteponer la palabra clave **base** un punto (.) y el nombre de las propiedades y para hacer referencia a la propiedad de la subclase, se hace de la misma forma, pero utilizado el comando **this**. La forma de indicar que un clase deriva de otra se utilizado el comando " : " en la declaración de la subclase. En C# solo existe herencia simple, es decir que una clase solo puede derivar de una clase.

Ejemplo 1: Ejercicio ilustrativo sobre herencia, redefinición de métodos y variable y método destructor.

```
public class ClaseA {           //Superclase A
    public int variable_x = 1;
    public int método_x() {
        return variable_x * 10;    }
    public int método_y() {
        return variable_x + 100; }    //El valor de variable_x es 1
    ~ClaseA() {                  //Destructor de clase A
        System.Console.WriteLine("Recursos de ClaseA liberados");    }
}
public class ClaseB : ClaseA {   //Subclase B
    int variable_x = 2;          //Redefinición del variable_x
    public int método_x() {      //Redefinición del Método_x
        return variable_x * -10;    }    //El valor de variable_x es 2
    public int método_z() {
        variable_x = base.variable_x + 3;    // El valor de base.variable_x es 1
        return base.método_x() + variable_x; } // El valor de super.método_x() es 10
    ~ClaseB () {                //Destructor de clase B
        System.Console.WriteLine("Recursos de ClaseB liberados");
    }
}
class Program    {    //Clase principal
    static void Main(string[] args)    {
        ClaseB objClaseB = new ClaseB();
        System.Console.WriteLine(objClaseB.variable_x);    //Escribe 2
        System.Console.WriteLine(objClaseB.método_y());    //Escribe 101 método heredado
        System.Console.WriteLine(objClaseB.método_x());    //Escribe -20
        System.Console.WriteLine(objClaseB.método_z());    //Escribe 14
        objClaseB = null;    //Destrucción del objeto objClaseB
        System.GC.Collect();
        System.Console.WriteLine("Presiones una tecla para terminar...");
        System.Console.ReadKey();
    }
}
```

Ejercicio 2: En este ejemplo se emplean 2 tipos de figuras geométricas (círculos y rectángulos). Consta de una superclase **Figura**: donde se declaran las variables miembro comunes para las 2 figuras: el centro manejado con un vector enteros de dos posiciones y el color con un string, los métodos constructores y los métodos para manipular las variables miembro. Una subclase **Círculo**: que declara la variable miembro radio que es particular para los círculos, los constructores y los métodos para manipular el radio. Una subclase **Rectángulo**: que declara las variables miembro lados manejadas con un vector de reales, los constructores y los métodos para manipular la variable lados. Y por último una clase **Program** para observar el funcionamiento de las clases anterior y es la ejecutable. El árbol de Herencia es:



```
public class Figura {    //Superclase
    private String colorFigura;
    private int[] posicion = new int[2]; // vector para manejar la posición o ubicación
    //Metodos Constructores
    public Figura() {
        colorFigura = "Blanco";
        posicion[0] = 0;
        posicion[1] = 0;
    }
    public Figura(String color)    {
        colorFigura = color;
        posicion[0] = 0;
        posicion[1] = 0;
    }
}
```

```

public Figura(String color, int[] posicion)    {
    colorFigura = color;
    this.posicion[0] = posicion[0];
    this.posicion[1] = posicion[1];
}
//Metodos para el manejo de variables (color y el centro)
public void setColor(String color) {
    colorFigura = color;
}
public String getColor() {
    return colorFigura;
}
public void setCentro(int[] posicion) {
    this.posicion[0] = posicion[0];
    this.posicion[1] = posicion[1];
}
public int[] getCentro() {
    return posicion;
}
}

public class Circulo : Figura { //Subclase Circulo que deriva de Figura
    private double radio;
    //Metodos Constructores para los Objetos Circulo
    public Circulo(double radio) { this.radio = radio; }

    public Circulo(double radio, string color) : base(color) {
        this.radio = radio;
    }
    public Circulo(double radio, string color, int[] posicion) : base(color, posicion) {
        this.radio = radio;
    }
    //Metodos para el manejo de el radio
    public void setRadio(double radio) {
        this.radio = radio;
    }
    public double getRadio() {
        return radio;
    }
}

public class Rectangulo : Figura { //Subclase Rectangulo que deriva de Figura
    private double lado1;
    private double lado2;
    //Metodos Constructores para los Objetos Rectangulo
    public Rectangulo(double lado1, double lado2) {
        this.lado1 = lado1;
        this.lado2 = lado2;
    }
    public Rectangulo(double lado1, double lado2, string color) : base(color) {
        this.lado1 = lado1;
        this.lado2 = lado2;
    }
    public Rectangulo(double lado1, double lado2, string color, int[] posicion) : base(color,
posicion) {
        this.lado1 = lado1;
        this.lado2 = lado2;
    }
    //Metodos para el manejo de la variable lado
    public void setLados(double lado1, double lado2) {
        this.lado1 = lado1;
        this.lado2 = lado2;
    }
    public double getLado1() {
        return lado1;
    }
}

```

```

    public double getLado2() {
        return lado2;
    }
}

class Program {
    //metodos sobre cargados imp : para imprimir
    static void imp(string s) { System.Console.WriteLine(s); //imprime una cadena
    }

    static void imp(Circulo c) { //imprime un objeto circulo
        imp("\nCIRCULO:");
        imp("El radio es: " + c.getRadio());
        int[] centro = c.getCentro();
        imp("El Centro es: (" + centro[0] + " , " + centro[1] + ")");
        imp("El Color es:" + c.getColor());
    }

    static void imp(Rectangulo r) { //imprime un objeto rectangulo
        imp("\nRECTANGULO:");
        double lado1 = r.getLado1();
        double lado2 = r.getLado2();
        imp("Los lados son: (" + lado1 + " , " + lado2 + ")");
        int[] posicion = r.getCentro();
        imp("El Centro es: (" + posicion[0] + " , " + posicion[1] + ")");
        imp("El Color es:" + r.getColor());
    }

    //metodo principal
    static void Main(string[] args) {
        int[] posicion = { 10, 20 }; //Declara el Vector posicion
        double lado1 = 50.0; //Declara los lados del rectangulo
        double lado2 = 100.0;
        //Construye un objetos circulo y despues uno rectangulo
        Circulo objcir = new Circulo(3.0, "Rojo");
        Rectangulo objrec = new Rectangulo(lado1, lado2, "Verde", posicion);
        imp("PROGRAMA SOBRE FIGURAS GEOMETICAS\n");
        imp(objcir); //Imprime Objeto circulo 1
        imp(objrec); //Imprime Objeto rectangulo 1

        //Método de la subclase Circulo
        imp("\nDigite el nuevo Radio");
        objcir.setRadio(Int32.Parse(Console.ReadLine()));
        imp("\nEl radio es: " + objcir.getRadio());
        //Método heredado del la clase figura
        int[] centro = objcir.getCentro();
        imp("El Centro es: [" + centro[0] + " , " + centro[1] + "]");
        Circulo objcir2 = new Circulo(5.7);
        imp(objcir2); //Imprime Objeto circulo 2

        imp("\nDigite el Primer lado");
        int Lado1 = Int32.Parse(Console.ReadLine());
        imp("\nDigite el segudo lado");
        int Lado2 = Int32.Parse(Console.ReadLine());
        objrec.setLados(Lado1, Lado2);
        imp("\nDigite el color deseado");
        string color = Console.ReadLine();
        objrec.setColor(color);
        Rectangulo objRec1 = new Rectangulo(Lado1, Lado2, color);
        imp(objRec1); //Imprime Objeto rectangulo 2

        imp("\nDigite el Primer lado");
        Lado1 = Int32.Parse(Console.ReadLine());
        imp("\nDigite el segudo lado");
        Lado2 = Int32.Parse(Console.ReadLine());
        Rectangulo objRec2 = new Rectangulo(Lado1, lado2);
        imp(objRec2); //Imprime Objeto rectangulo 3
    }
}

```



```

        imp("\nDigite el Primer lado");
        Lado1 = Int32.Parse(Console.ReadLine());
        imp("\nDigite el segundo lado");
        Lado2 = Int32.Parse(Console.ReadLine());
        // objrec.setLados(Lado1, Lado2);
        imp("\nDigite el color deseado");
        color = Console.ReadLine();
        objrec.setColor(color);
        int[] Centro = objrec.getCentro();
        Rectangulo objRec3 = new Rectangulo(Lado1, lado2, color, Centro);
        imp(objRec3); //Imprime Objeto rectangulo 2

        Console.WriteLine("\n\tPresione una tecla para continuar...");
        Console.ReadKey();
    }
}

```

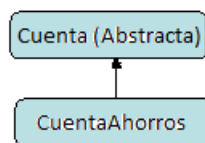


Práctica 3: Realizar un programa para una editorial que publica libros y periódicos, en el programa se deben manejar las siguientes variables: Número de hojas, valor hoja, hojas a color, titulo, tipo de portada, nombre autor, nombre periódico y fecha. Emplear una superclase publicaciones, dos subclases libro y periódico y la clase Leer para ingresar la información. La clase de prueba o editorial debe crear objetos tanto libro como periódico y calcular el costo que se obtiene de: el valor de la hoja, el tipo de portada (sus valores son lujo = 12.000\$, normal = 7.000\$ y económica = 4.000\$) y si es a color cada hoja vales 300\$ más que a blanco y negro.

6.2. Abstracción (Clase Abstracta)

Una clase Abstracta es una clase particular de la que no se pueden crear objetos. Su utilidad es permitir que otras clases deriven de ella, proporcionándoles un marco o modelo que deben seguir y algunos métodos de utilidad general. Las clases abstractas se declaran anteponiéndoles la palabra **abstract** al comando **class**, Son las únicas que puede tener métodos declarados como abstracto, en cuyo caso no se da definición del método, es decir no poseen cuerpo. Si una clase tiene algún método **abstract** es obligatorio que la clase sea **abstract**. En cualquier subclase este método deberá ser redefinido para poder crear objetos de ella.

Ejemplo: Consta de 3 clases: La clase abstracta **Cuenta**: que agrupa los datos comunes a cualquier tipo de cuenta bancaria. La subclase **CuentaAhorro**: clase derivada de Cuenta y es el prototipo para hacer los objetos. Y **TestCuenta** (ejecturable): Aplicación para trabajar y crear los objetos de la clase **CuentaAhorros**



6.3. Polimorfismo

Consigue que un mismo método pueda actuar sobre diferentes tipos de objetos y comportarse de modo diferente. El polimorfismo es el comportamiento diferente de un objeto dependiendo del contexto en que se ejecuta. Para implementarlo se hace uso combinado de la herencia y la abstracción.

Ejemplo: Este es el mismo ejercicio desarrollado en el Ejemplo 2 de Herencia sobre figuras geométricas, pero aplicando el concepto de polimorfismo. Para demostrar el polimorfismo se adjuntan los métodos de área y perímetro. Las clase son: **Figura**; clase abstracta, con los métodos abstractos área y perímetro, **Rectángulo** y **Circulo**; que heredan de **Figura** e implementan los 2 métodos abstractos, pero cada una dependiendo de la figura específica, **PruebaFiguras**; clase principal o ejecutable.

6.4. Otros Conceptos

Paquetes

Sirven para agrupar clase relacionadas, las clases dentro de un paquete deben tener nombres diferentes. Para declarar un paquete basta con introducir la instrucción `namespace` NombrePaquete después de las importaciones o `using`. Por ejemplo, si deseamos crear el paquete banco con las tres clases del ejemplo anterior, se coloca la sentencia: `namespace Banco`. En C# cada que se crea una solución o proyecto .Net por defecto le coloca el mismo nombre al paquete, pero si se desea se le puede cambiar. Una solución puede estar integrada por varios proyectos, pero solo uno debe ser el principal o el que inicia la ejecución.

Para hacer uso de una clase de otro paquete hay que importarla con el comando: `using` NombrePaquete; y si necesitan una clase específica de dicho paquete, a continuación se coloca su nombre, el formato es: `using` NombrePaquete.NombreClase;. Se pueden importar tantos paquetes como se requieran.

Interface

Es una especie de clase con un conjunto de declaraciones de métodos sin definición o abstractos. Una interface define un tipo de conducta o construcción. Todas las clases que implementan una interface están obligadas a proporcionar una definición (código) de los métodos de la interface, y en ese sentido adquieren una conducta o modo de funcionamiento.

Una clase puede implementar una o varias interfaces. Para indicar que una clase implementa una o más interfaces se ponen los nombres de las interfaces, separados por comas, detrás del comando ":", como cuando se hereda. Formato: [Acceso] `class` NombreClase : NombreInterface1,... NombreInterfaceN { ...}

Para crear una interface se define de un modo muy similar a las clases. Formato:

[Acceso] `interface` Nombreinterface [: NombreInterface1, NombreInterface2 { ...}

Ejemplo: Sencillo de implementación explícita de Interface para el manejo de la clase DateTime

```
interface IFecha { // Interface
    //Métodos solo declarados
    int dia();
    int mes();
    int ano();
}
public class Fecha : IFecha //clase que implemente la interface
{
    DateTime fecha;
    public Fecha(DateTime actual) { //Metodo constructor
        fecha = actual;
    }

    int IFecha.dia() { //Redefinicion del metodo dia de la interface
        return fecha.Day;
    }
    int IFecha.mes() { //Redefinicion del metodo mes de la interface
        return fecha.Month;
    }
    int IFecha.ano() { //Redefinicion del metodo año de la interface
        return fecha.Year;
    }
}

class Program { //Clase ppal
```

```

static void imp(string men) {
    System.Console.WriteLine(men);
}
static void Main(string[] args)
{
    DateTime fecha = DateTime.Now; //Captura la fecha del sistema
    Fecha actual = new Fecha(fecha); //Crea un objeto de la clase fecha
    IFecha hoy =(IFecha) actual;    //Crea un objeto de la interface
    imp ("la fecha actual es Dia: "+ hoy.dia() +" Mes: "+hoy.mes()+
        " Y el año: "+hoy.ano());
    System.Console.ReadKey();
}
}

```

Anexos:

1. Esta es la clase de propósito general Leer que se utiliza para el ingreso de cualquier tipo de información desde el teclado.

```

public class Leer
{
    //Método para imprimir cadenas.
    public static void imp(String mensaje)
    {
        System.Console.Write(mensaje);
    }
    //Método para leer cadenas.
    public static string dato()
    {
        string sdato = "";
        try
        {
            sdato = System.Console.ReadLine();
        }
        catch (Exception e)
        {
            imp("Error: " + e.ToString() + "\n");
        }
        return sdato;    // devolver la cadena tecleada
    }
    //Método para leer un caracter.
    public static char caracter()
    {
        char car = '\0';
        try
        {
            car = (char)System.Console.Read();
            System.Console.ReadLine(); ////Limpia el resto de caracteres del flujo de entrada
        }
        catch (Exception e)
        {
            imp("Error: " + e.Message + "\n");
        }
        return car;    // devolver el caracter tecleado
    }
}

```

```

//Método para leer un entero pequeño byte.
public static byte datoByte()
{
    try
    {
        string sdato = dato(); //Llama al metodo dato para que lea.
        if (sdato == null)
        {
            //Pregunta si no se digito nada
            imp("No se ingreso ningun valor => 0\n");
            return 0;
        }
        else return Byte.Parse(sdato); //devolver el dato tecleado
    }
    catch (Exception e)
    {
        imp(e.Message + " es dato no valido... Teclee otro: ");
        return datoByte();
    }
    //Vuelve a llamar a la función datoShor
}

//Método para leer un entero corto.
public static short datoShort()
{
    try
    {
        string sdato = dato(); //Llama al metodo dato para que lea.
        if (sdato == null)
        {
            //Pregunta si no se digito nada
            imp("No se ingreso ningun valor => 0\n");
            return 0;
        }
        else return Int16.Parse(sdato); //devolver el dato tecleado
    }
    catch (Exception e)
    {
        imp(e.Message + " es dato no valido... Teclee otro: ");
        return datoShort();
    }
    //Vuelve a llamar a la función datoShor
}

//Método para leer un entero .
public static int datoInt()
{
    try
    {
        string sdato = dato();
        if (sdato == null)
        {
            //Pregunta si no se digito nada
            imp("No se ingreso ningun valor => 0\n");
            return 0;
        }
        else return Int32.Parse(sdato); //devolver el dato tecleado
    }
    catch (Exception e)
    {
        imp(e.Message + " es dato no valido... Teclee otro: ");
        return datoInt();
    }
    //Vuelve a llamar a la función datoInt
}

//Método para leer un entero largo.
public static long datoLong()
{
    try
    {
        String sdato = dato();

```

```

        if (sdato == null)
        {
            //Pregunta si no se digito nada
            imp("No se ingreso ningun valor => 0\n");
            return 0;
        }
        else return Int64.Parse(sdato); //devolver el dato tecleado
    }
    catch (Exception e)
    {
        imp(e.Message + " es dato no valido... Teclee otro: ");
        return datoLong();
    } //Vuelve a llamar a la función datoLong
}
//Método para leer un real corto.
public static float datoFloat()
{
    try
    {
        string sdato = dato();
        if (sdato == null)
        {
            //Pregunta si no se digito nada
            imp("No se ingreso ningun valor => 0.0\n");
            return 0.0f;
        }
        else return Single.Parse(sdato); //devolver el dato tecleado
    }
    catch (Exception e)
    {
        imp(e.Message + " es dato no valido... Teclee otro: ");
        return datoFloat();
    } //Vuelve a llamar a la función datoFloat
}
//Método para leer un real largo.
public static double datoDouble()
{
    try
    {
        string sdato = dato();
        if (sdato == null)
        {
            //Pregunta si no se digito nada
            imp("No se ingreso ningun valor => 0.0\n");
            return 0.0;
        }
        else return Double.Parse(sdato); //devolver el dato tecleado
    }
    catch (Exception e)
    {
        imp(e.Message + " es dato no valido... Teclee otro: ");
        return datoDouble();
    }
}
}
}

```